

Objetivo do Plano

Ao final de 30 dias: Construir uma compreensão sólida dos pilares do Backend Moderno. Você será capaz de projetar e implementar APIs RESTful básicas, entender a arquitetura de microsserviços, utilizar Docker para containerização, realizar deploy de aplicações e compreender o papel da mensageria em sistemas distribuídos.



Cronograma de Estudos (4 Semanas)

Semana 1: Fundamentos de APIs REST e Arquitetura

Foco: Dominar a base de como os serviços se comunicam na web.

- Dia 1-2: Fundamentos de HTTP e REST
 - Verbos HTTP (GET, POST, PUT, DELETE, PATCH).
 - Codes de Status (2xx, 3xx, 4xx, 5xx).
 - Estrutura de uma API RESTful (Stateless, Resources).
 - *Prática*: Desenhar um diagrama de fluxo para uma API simples (ex: CRUD de usuários).
- Dia 3-4: Design de APIs
 - Versionamento de APIs (URI vs Header).
 - Tratamento de Erros e Logging.
 - Autenticação básica (API Keys, Intro a OAuth2/JWT).
 - *Prática*: Documentar uma API fictícia usando OpenAPI (Swagger).
- Dia 5-7: Mão na Massa (API Simples)
 - Escolha uma linguagem (Node.js, Python, Go, Java).
 - Crie uma API RESTful simples conectada a um banco de dados SQL ou NoSQL.
 - Teste com Postman ou Insomnia.

 Recursos do Seu Contexto: *Procure nos seus e-mails/Drive por*: PDFs sobre "Arquitetura REST", "HTTP Fundamentals", ou tutoriais de "Build API [Sua Linguagem]".

- [Inserir link/arquivo de tutorial REST que você já tem]
 - [Inserir link/arquivo de documentação interna ou anotações]
-

Semana 2: Containerização com Docker

Foco: Empacotar sua aplicação para rodar em qualquer lugar.

- Dia 8-9: Conceitos de Containers vs VMs
 - O que é um Container?
 - Arquitetura do Docker.
 - Instalação e primeiros comandos (`docker run, ps, stop, images`).
- Dia 10-12: Dockerfile e Imagens
 - Escrevendo seu primeiro `Dockerfile` (FROM, RUN, CMD, COPY).
 - Entendendo camadas (layers) e cache.
 - Melhores práticas para imagens leves (ex: usar Alpine Linux).
 - *Prática*: Criar uma imagem Docker para a API da Semana 1.
- Dia 13-14: Docker Compose e Redes
 - Orquestração local com `docker-compose.yml`.
 - Criando Redes Docker.
 - Volumes e persistência de dados.
 - *Prática*: Subir sua API + Banco de Dados usando apenas Docker Compose.

 Recursos do Seu Contexto: *Procure por*: Guias de "Docker Best Practices", "Docker Compose examples", ou cheatsheets de Docker.

- [Inserir anotações sobre Docker que você já possui]
 - [Inserir vídeo ou tutorial de containerização salvo]
-

Semana 3: Introdução a Microsserviços e Mensageria

Foco: Escalar a arquitetura e comunicação assíncrona.

- Dia 15-16: Monólito vs Microsserviços
 - Quando quebrar um monólito?
 - Desafios: latência, consistência de dados, deploy.
 - Padrões comuns (API Gateway, Service Discovery).
- Dia 17-19: Mensageria (Message Brokers)
 - Por que usar filas? (Desacoplamento, Resiliência).
 - Conceitos: Producer, Consumer, Broker, Tópicos vs Filas.
 - Introdução a ferramentas: RabbitMQ ou Apache Kafka.
 - *Prática*: Criar um serviço produtor e um consumidor simulando um processo de "Pedido de Compra".

- Dia 20-21: Comunicação Síncrona vs Assíncrona
 - Quando usar REST vs Mensageria.
 - Padrão Saga (básico) para transações distribuídas.
 - *Prática*: Integrar a API da Semana 1 com a fila da Mensageria.

 Recursos do Seu Contexto: *Procure por*: Artigos sobre "Event Driven Architecture", "Kafka vs RabbitMQ", ou estudos de caso de migração para microsserviços.

- [Inserir whitepaper ou artigo sobre microsserviços]
 - [Inserir slide ou doc sobre mensageria]
-

Semana 4: Cloud e Deploy (O "Pulo do Gato")

Foco: Colocar sua aplicação no mundo real.

- Dia 22-23: Conceitos de Cloud Computing
 - IaaS vs PaaS vs SaaS.
 - Proveedores principais: AWS, Azure, GCP (escolha um para focar).
 - Serviços essenciais: Compute (EC2, Lambda), Banco de Dados (RDS, Firestore), Storage (S3).
- Dia 24-25: Estratégias de Deploy
 - Deploy manual vs CI/CD (Integração Contínua/Entrega Contínua).
 - Conceitos de Pipeline (Build -> Teste -> Deploy).
 - Ferramentas: GitHub Actions ou GitLab CI.
- Dia 26-28: Projeto Final Integrador
 - Desafio: Refatore o projeto da Semana 1.
 - Empacote o projeto com Docker.
 - Crie um pipeline básico no GitHub Actions.
 - Faça o deploy em uma conta gratuita de Cloud (ex: AWS Free Tier, Render, ou Railway).
 - Conecte com um serviço de mensageria na nuvem (ex: AWS SQS ou RabbitMQ cloud).
- Dia 29-30: Revisão e Próximos Passos
 - Revisão de conceitos difíceis.
 - O que estudar agora? (Kubernetes, Terraform, Observabilidade).
 - Atualizar o portfólio/GitHub com o projeto final.

 Recursos do Seu Contexto: *Procure por*: Certificações de Cloud (AWS Certified Cloud Practitioner, etc.), tutoriais de "Deploy na AWS/Azure", ou configurações de CI/CD que você já tenha visto.

- [Inserir link de curso de Cloud que você tem]
 - [Inserir exemplo de pipeline YAML]
-



Dicas Finais

1. Não decore, entenda: O foco é entender *por que* usamos Docker ou Mensageria, não apenas o comando.
2. Git é seu amigo: Committe seu código a cada dia. Isso cria um portfólio e te ajuda a recuperar versões.
3. Anote tudo: Crie um arquivo `README.md` no seu projeto com o que você aprendeu a cada semana.
4. Use seus materiais: Se você tem cursos, PDFs ou anotações no seu Drive, priorize-os. Eles já foram selecionados por você ou pela sua empresa.

Se precisar de ajuda com algum conceito específico (ex: "Como funciona o handshake do HTTP?" ou "Qual a diferença entre Kafka e RabbitMQ?"), é só perguntar!